TUTORIAL 5 – MEMORY PROFILING WITH VALGRIND

1. Objectives:

- Measure memory consumption of processes
- Detect memory leak

In this tutorial, we will explore how to use **Valgrind** to profile memory in C and C++ programs. Valgrind is a powerful tool for detecting memory-related issues, such as:

- **Invalid memory accesses**: Accessing memory that has already been freed or accessing memory outside the bounds of allocated memory.
- **Use of uninitialized memory**: Using variables or pointers that have not been initialized.
- **Memory leaks**: Memory that is allocated but never freed, leading to wasted memory and potential crashes.

We will go through how to use Valgrind effectively to detect these types of errors and analyze their causes.

2. What is Valgrind?

Valgrind is an instrumentation framework that allows you to build dynamic analysis tools. One of its most useful tools is **Memcheck**, which helps to detect memory errors in programs written in C and C++.

The **Memcheck** tool in Valgrind can detect the following types of errors:

- **Invalid memory accesses**: This occurs when your program tries to read or write to memory that has already been freed or is outside the bounds of an allocated block.
- **Uninitialized memory usage**: This happens when you use a pointer or variable that has not been properly initialized.
- **Memory leaks**: This happens when memory is allocated dynamically but not freed, resulting in memory that can't be reclaimed, which may eventually exhaust available memory.

3. Creating a Program with Memory Issues

Let's write a C program that contains the following common memory errors:

- **Invalid memory access**: Writing to memory outside the allocated space or accessing memory that has already been freed.
- **Use of uninitialized memory**: Using variables or pointers that have not been initialized.
- **Memory leak**: Allocating memory but failing to free it.

Sample C Program (memory_error.c):

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  // Allocate memory for 5 integers
  int *ptr1 = malloc(5 * sizeof(int));
  if (ptr1 == NULL) {
    printf("Memory allocation failed!\n");
    return 1;
  }

  // Invalid memory access: out-of-bounds write
  ptr1[5] = 42;  // Accessing ptr1[5] is invalid (valid indices are 0 to 4)

  // Use of uninitialized memory
  int *ptr2;  // ptr2 is declared but not initialized
  printf("Uninitialized value: %d\n", *ptr2);  // Dereferencing an uninitialized pointer

  // Memory leak: ptr1 is allocated but never freed
  // No call to free(ptr1)

  // Allocate additional memory for another pointer but forget to free it
  int *ptr3 = malloc(10 * sizeof(int));
  // No call to free(ptr3)
```

```
    return 0;
}
```

Explanation of Errors in the Program:

1. **Invalid Memory Access**:
    a. The program allocates memory for 5 integers but then tries to write to ptr1[5], which is out of bounds (valid indices are 0 to 4).

```
ptr1[5] = 42;  // Invalid memory access: out-of-bounds write
```

2. **Use of Uninitialized Memory**:
    a. The pointer ptr2 is declared but never initialized. Dereferencing an uninitialized pointer results in undefined behavior.

```
int *ptr2;
printf("Uninitialized value: %d\n", *ptr2);  // Dereferencing uninitialized pointer
```

3. **Memory Leak**:
    a. The program allocates memory for both ptr1 and ptr3, but neither is freed before the program exits. This results in **memory leaks** because the memory is still allocated when the program terminates.

```
free(ptr1);  // Missing
free(ptr3);  // Missing
```

4. Compiling the Program

To ensure that Valgrind can provide the most detailed output, we need to compile the program with debugging symbols. This will allow Valgrind to report the exact lines of code where the errors occur.

```
gcc -g -o memory_error memory_error.c
```

The -g flag includes debugging information in the executable, which helps Valgrind trace errors back to specific lines in the source code.

5. Running the Program with Valgrind

Now that the program is compiled, we can use **Valgrind** to detect the memory issues. To do so, use the following command:

```
valgrind --leak-check=full --track-origins=yes --show-leak-kinds=all ./memory_error
```

Explanation of Valgrind Options:

- --leak-check=full: Tells Valgrind to perform a full check for memory leaks, including details about each leak.
- --track-origins=yes: Tracks the origin of uninitialized memory and reports where uninitialized values were created.
- --show-leak-kinds=all: Displays all types of leaks, including "definitely lost," "indirectly lost," etc.

6. Valgrind Output

When you run the program with Valgrind, you will see output similar to the following:

```
==12345== Memcheck, a memory error detector
==12345== Copyright (C) 2002-2023, and GNU GPL'd, by Julian Seward et al.
==12345== Using Valgrind-3.20.0 and LibVEX; rerun with -h for copyright info
==12345== Command: ./memory_error
==12345==
==12345== Invalid write of size 4
==12345==    at 0x4005F7: main (memory_error.c:12)
==12345==  Address 0x1ffefffff is 1 bytes after a block of size 20 alloc'd
==12345==    at 0x4C2E57F: malloc (vg_replace_malloc.c:299)
==12345==    by 0x4005D6: main (memory_error.c:10)
==12345==
==12345== Use of uninitialized value of size 4
==12345==    at 0x4005E2: main (memory_error.c:16)
==12345==  Uninitialized value was created by a stack allocation
==12345==    at 0x4005D2: main (memory_error.c:14)
==12345==
```

```
==12345== 20 bytes in 1 blocks are definitely lost in loss record 1 of 2
==12345==    at 0x4C2E57F: malloc (vg_replace_malloc.c:299)
==12345==    by 0x4005D6: main (memory_error.c:10)
==12345==
==12345== 40 bytes in 1 blocks are definitely lost in loss record 2 of 2
==12345==    at 0x4C2E57F: malloc (vg_replace_malloc.c:299)
==12345==    by 0x4005D6: main (memory_error.c:19)
==12345==
==12345== LEAK SUMMARY:
==12345==    definitely lost: 60 bytes in 2 blocks
==12345==    indirectly lost: 0 bytes in 0 blocks
==12345==    possibly lost: 0 bytes in 0 blocks
==12345==    still reachable: 0 bytes in 0 blocks
==12345==    suppressed: 0 bytes in 0 blocks
==12345==
==12345== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
```

Key Sections of the Output:

1. **Invalid Memory Access**:

```
==12345== Invalid write of size 4
==12345==    at 0x4005F7: main (memory_error.c:12)
==12345==  Address 0x1ffeffff is 1 bytes after a block of size 20 alloc'd
```

Valgrind reports an out-of-bounds write to ptr1[5], which is invalid.

2. **Use of Uninitialized Memory**:

```
==12345== Use of uninitialized value of size 4
==12345==    at 0x4005E2: main (memory_error.c:16)
==12345==  Uninitialized value was created by a stack allocation
```

Valgrind detects the use of the uninitialized pointer ptr2 in the printf function.

3. **Memory Leak**:

less
Copy code
==12345== 20 bytes in 1 blocks are definitely lost in loss record 1 of 2
==12345== at 0x4C2E57F: malloc (vg_replace_malloc.c:299)
==12345== by 0x4005D6: main (memory_error.c:10)

Valgrind reports that memory allocated by malloc for ptr1 and ptr3 is **definitely lost**, meaning the memory was not freed before the program exited.

7. TODO jobs: fixing the errors

To fix the errors detected by Valgrind, you are asked to:

1. **Correct Invalid Memory Access**: Make sure you access memory within the bounds of the allocated array.
2. **Initialize Pointers/Variables**: Always initialize pointers and variables before using them.
3. **Free Allocated Memory**: Always free memory after you're done using it to prevent memory leaks.

You are asked to provide the corrected program following the above guidance. Also explain codes that you add and/or revise.

Corrected Program:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  // Allocate memory for 5 integers
  int *ptr1 = malloc(5 * sizeof(int));
  if (ptr1 == NULL) {
    printf("Memory allocation failed!\n");
    return 1;
  }

  // Corrected memory access: Access within bounds
  ptr1[4] = 42;

  // Corrected uninitialized memory usage
  int *ptr2 = malloc(sizeof(int)); // Initialize ptr2
  *ptr2 = 0;  // Initialize memory pointed to by ptr2
  printf("Initialized value: %d\n", *ptr2);  // Safe to use

  // Free the allocated memory to avoid memory leaks
  free(ptr1);
  free(ptr2);  // Free ptr2

  // Allocate additional memory for another pointer and free it
  int *ptr3 = malloc(10 * sizeof(int));
  free(ptr3);  // Freeing ptr3

  return 0;
}
```

Conclusion

Valgrind is an essential tool for identifying and fixing memory-related issues in C and C++ programs. It helps you catch errors like invalid memory accesses, use of uninitialized variables, and memory leaks that can otherwise go unnoticed. By following the steps in this tutorial, you can use Valgrind to improve the robustness of your programs and prevent common memory-related bugs.